

Contents

Python Coding Interview Preparation Guide	1
Table of Contents	1
Introduction	2
How to Approach Coding Interviews	2
Problem-Solving Framework	2
Common Python Libraries and Built-in Functions	3
Collections Module	3
Heapq Module	4
Itertools Module	4
Built-in Functions	5
Other Useful Tools	5
Arrays and Hashing	5
Theory	5
Example Problems	7
Two Pointers	10
Theory	10
Example Problems	12
Stack and Queue	21
Theory	21
Example Problems	23
Example Problems	29
Example Problems	35
Trees	39
Theory	39
Example Problems	41
Example Problems	47
Trie Data Structure	51
Theory	51
Example Problems	53
Graphs	59
Theory	59
Example Problems	63
Backtracking	68
Theory	68
Example Problems	69

Python Coding Interview Preparation Guide

Table of Contents

1. Introduction
 - How to Approach Coding Interviews
 - Problem-Solving Framework

2. Common Python Libraries and Built-in Functions
3. Arrays and Hashing
4. Two Pointers
5. Sliding Window
6. Stack and Queue
7. Binary Search
8. Linked Lists
9. Trees
10. Heaps/Priority Queues
11. Trie Data Structure
12. Graphs
13. Backtracking
14. Greedy Algorithms
15. Dynamic Programming
16. Intervals
17. Math and Geometry

Introduction

How to Approach Coding Interviews

Coding interviews can be stressful, but with proper preparation and a systematic approach, you can perform well. Here are some key strategies:

1. **Understand the problem completely:**
 - Ask clarifying questions
 - Confirm your understanding with examples
 - Discuss edge cases early
2. **Think out loud:**
 - Share your thought process
 - Explain trade-offs you're considering
 - Discuss alternative approaches
3. **Start with a simple solution:**
 - Begin with a brute force approach to show you can solve the problem
 - Optimize iteratively
 - Explain each optimization step clearly
4. **Test your solution:**
 - Walk through your code with examples
 - Pay special attention to edge cases
 - Fix bugs methodically
5. **Analyze complexity:**
 - Be prepared to discuss time and space complexity
 - Know the complexity of standard operations

Problem-Solving Framework

For most interview problems, follow this framework:

1. **Clarify the problem:**
 - What are the inputs and expected outputs?
 - What constraints exist?
 - Are there any performance requirements?
2. **Work through examples:**
 - Use simple examples to understand the problem
 - Try edge cases to identify potential issues
3. **Formulate an approach:**
 - Consider which data structures or algorithms might be appropriate
 - Think about the problem in terms of patterns you recognize
4. **Implement the solution:**
 - Write clean, readable code
 - Use descriptive variable names
 - Structure your code logically
5. **Test and debug:**
 - Test with your examples
 - Identify and fix any issues
 - Consider any optimizations

Common Python Libraries and Built-in Functions

Python's standard library contains many powerful tools that can simplify your solutions during coding interviews. Here are some of the most useful:

Collections Module

```
from collections import deque, Counter, defaultdict, OrderedDict
```

```
# deque: Efficient append/pop from both ends
```

```
queue = deque([1, 2, 3])
queue.append(4)           # Add to right: [1, 2, 3, 4]
queue.appendleft(0)       # Add to left: [0, 1, 2, 3, 4]
queue.pop()               # Remove from right: [0, 1, 2, 3]
queue.popleft()           # Remove from left: [1, 2, 3]
```

```
# Counter: Count occurrences
```

```
counts = Counter("mississippi") # {'i': 4, 's': 4, 'p': 2, 'm': 1}
most_common = counts.most_common(2) # [('i', 4), ('s', 4)]
```

```
# defaultdict: Dictionary with default values
```

```
word_groups = defaultdict(list)
words = ["eat", "tea", "tan", "ate", "nat", "bat"]
for word in words:
    sorted_word = "".join(sorted(word))
    word_groups[sorted_word].append(word)
# defaultdict(<class 'list'>, {'aet': ['eat', 'tea', 'ate'], 'ant': ['tan', 'nat'], 'abt':
```

```
# OrderedDict: Dictionary that remembers insertion order (less important since Python 3.7)
ordered = OrderedDict()
ordered['first'] = 1
ordered['second'] = 2
```

Heapq Module

```
import heapq

# Create a heap from a list
nums = [3, 1, 4, 1, 5, 9]
heapq.heapify(nums) # [1, 1, 4, 3, 5, 9]

# Push item to heap
heapq.heappush(nums, 2) # [1, 1, 2, 3, 5, 9, 4]

# Pop smallest item
smallest = heapq.heappop(nums) # smallest = 1, nums = [1, 3, 2, 4, 5, 9]

# Get n smallest/largest elements
smallest_three = heapq.nsmallest(3, [3, 1, 4, 1, 5, 9]) # [1, 1, 3]
largest_three = heapq.nlargest(3, [3, 1, 4, 1, 5, 9]) # [9, 5, 4]
```

Itertools Module

```
import itertools

# combinations: All possible ways to select k items
combinations = list(itertools.combinations([1, 2, 3, 4], 2))
# [(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)]

# permutations: All possible arrangements of k items
permutations = list(itertools.permutations([1, 2, 3], 2))
# [(1, 2), (1, 3), (2, 1), (2, 3), (3, 1), (3, 2)]

# product: Cartesian product
product = list(itertools.product([1, 2], ['a', 'b']))
# [(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')]

# groupby: Group consecutive items
numbers = [1, 1, 1, 2, 2, 3, 3, 3, 3]
groups = [(k, list(g)) for k, g in itertools.groupby(numbers)]
# [(1, [1, 1, 1]), (2, [2, 2]), (3, [3, 3, 3, 3])]
```

Built-in Functions

```
# all() and any()
all([True, True, False]) # False
any([True, True, False]) # True

# map() and filter()
squared = list(map(lambda x: x**2, [1, 2, 3, 4])) # [1, 4, 9, 16]
evens = list(filter(lambda x: x % 2 == 0, [1, 2, 3, 4])) # [2, 4]

# zip(): Combine iterables
names = ["Alice", "Bob", "Charlie"]
ages = [25, 30, 35]
combined = list(zip(names, ages)) # [('Alice', 25), ('Bob', 30), ('Charlie', 35)]

# enumerate(): Get index and value
for i, name in enumerate(names):
    print(f"Index {i}: {name}")

# sorted() with key function
words = ["banana", "apple", "cherry"]
by_length = sorted(words, key=len) # ['apple', 'banana', 'cherry']
```

Other Useful Tools

```
# List Comprehensions
squares = [x**2 for x in range(10) if x % 2 == 0] # [0, 4, 16, 36, 64]

# Dictionary Comprehensions
square_map = {x: x**2 for x in range(5)} # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# Set Comprehensions
even_squares = {x**2 for x in range(10) if x % 2 == 0} # {0, 4, 16, 36, 64}

# bisect: Binary search functions
import bisect
sorted_list = [1, 3, 5, 7, 9]
insert_point = bisect.bisect_left(sorted_list, 4) # 2
bisect.insort_left(sorted_list, 4) # [1, 3, 4, 5, 7, 9]
```

Arrays and Hashing

Theory

Arrays and hash tables (dictionaries in Python) are fundamental data structures used in many algorithms. Understanding when and how to use them effectively is crucial for solving many coding interview problems.

Arrays Arrays in Python are implemented as dynamic lists that automatically resize as elements are added or removed.

Key Operations and Complexity: - Access by index: $O(1)$ - Append/Pop from end: $O(1)$ amortized - Insert/Delete at beginning or middle: $O(n)$ - Search for element: $O(n)$ - Slice: $O(k)$ where k is the length of the slice

Array Visualization:

```
0  1  2  3  4  5  ← indices
5  2  9  1  7  6  ← values
```

Common Array Patterns: 1. **Two-pass technique:** Process the array once to gather information, then process it again to solve the problem 2. **Prefix sums:** Pre-compute cumulative sums to enable $O(1)$ range sum queries 3. **Sorting:** Many problems become easier after sorting the array 4. **In-place modification:** Modify the array without using extra space

Hash Tables (Dictionaries) Hash tables provide efficient lookup, insertion, and deletion operations using a key-value mapping.

Key Operations and Complexity: - Lookup: $O(1)$ average, $O(n)$ worst-case - Insert/Delete: $O(1)$ average, $O(n)$ worst-case - Iterate: $O(n)$

Hash Table Visualization:

```
"apple" → 5
"banana" → 3
"cherry" → 8
"date" → 1
```

Common Hashing Patterns: 1. **Counting elements:** Using a dictionary to count occurrences 2. **Grouping elements:** Mapping values to groups with similar properties 3. **Memorization:** Storing computed results for quick lookup 4. **Two-Sum pattern:** Finding pairs with specific relationships

When to Use Arrays and Hashing Use arrays when: - You need ordered elements with constant-time access by index - You're performing operations from the end of the collection - You need to efficiently iterate through all elements in order

Use hash tables when: - You need to quickly check if an element exists - You need to associate values with keys - You're looking for patterns or duplicates - You need to count occurrences of elements - You're implementing a cache

Example Problems

Easy: Two Sum Problem: Given an array of integers `nums` and an integer `target`, write a function that returns the indices of any two numbers such that they add up to `target`. Any two indices satisfying this condition are valid solutions. If no solution exists, return an empty array.

Examples: - `two_sum([2, 7, 11, 15], 9)` should return `[0, 1]` (since `nums[0] + nums[1] = 2 + 7 = 9`) - `two_sum([3, 2, 4], 6)` should return `[1, 2]` (since `nums[1] + nums[2] = 2 + 4 = 6`) - `two_sum([3, 3], 6)` should return `[0, 1]` (since `nums[0] + nums[1] = 3 + 3 = 6`) - `two_sum([1, 2, 3], 7)` should return `[]` (no solution exists)

Approach: We can use a hash table to store the value and index of each number we've seen. For each number, we check if its complement (`target - num`) exists in the hash table.

```
def two_sum(nums: list[int], target: int) -> list[int]:
    seen = {} # Value -> Index

    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i

    return [] # No solution found
```

Visual Example:

`nums = [2, 7, 11, 15], target = 9`

Iteration 1:

Current number: 2 (index 0)
Complement: $9 - 2 = 7$
`seen = {2: 0}`

Iteration 2:

Current number: 7 (index 1)
Complement: $9 - 7 = 2$
2 is in seen! Return `[seen[2], 1] = [0, 1]`

Time Complexity: $O(n)$ - We traverse the array once Space Complexity: $O(n)$ - In the worst case, we store all elements in the hash table

Medium: Group Anagrams Problem: Given an array of strings `strs`, group the anagrams together and return the groups as a list of lists. An anagram is a word formed by rearranging the letters of another word, using all the original letters exactly once.

Examples: - `group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"])` should return `[["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]` (the order of the groups and the order of strings within groups can vary)
 - `group_anagrams([""])` should return `[[""]]` - `group_anagrams(["a"])` should return `[["a"]]`

Approach: We'll use a hash table where the key is a sorted version of each string (so anagrams have the same key), and the value is a list of anagrams.

```
def group_anagrams(strs: list[str]) -> list[list[str]]:
    anagram_groups = {}

    for s in strs:
        # Sort the string to use as a key
        sorted_str = ''.join(sorted(s))

        # Add to existing group or create a new one
        if sorted_str in anagram_groups:
            anagram_groups[sorted_str].append(s)
        else:
            anagram_groups[sorted_str] = [s]

    # Return the groups
    return list(anagram_groups.values())
```

Visual Example:

```
strs = ["eat", "tea", "tan", "ate", "nat", "bat"]
```

```
"eat" -> sorted: "aet" -> anagram_groups = {"aet": ["eat"]}
"tea" -> sorted: "aet" -> anagram_groups = {"aet": ["eat", "tea"]}
"tan" -> sorted: "ant" -> anagram_groups = {"aet": ["eat", "tea"], "ant": ["tan"]}
"ate" -> sorted: "aet" -> anagram_groups = {"aet": ["eat", "tea", "ate"], "ant": ["tan"]}
"nat" -> sorted: "ant" -> anagram_groups = {"aet": ["eat", "tea", "ate"], "ant": ["tan", "nat"]}
"bat" -> sorted: "abt" -> anagram_groups = {"aet": ["eat", "tea", "ate"], "ant": ["tan", "nat"], "abt": ["bat"]}
```

Result: `[["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]`

Time Complexity: $O(n * k \log k)$ where n is the number of strings and k is the maximum length of a string
 Space Complexity: $O(n * k)$ for storing all strings

Medium: Longest Consecutive Sequence Problem: Given an unsorted array of integers `nums`, write a function that returns the length of the longest

consecutive elements sequence. The sequence must be strictly increasing by 1 between consecutive elements.

Examples: - `longest_consecutive([100, 4, 200, 1, 3, 2])` should return 4 (the longest consecutive sequence is [1, 2, 3, 4]) - `longest_consecutive([0, 3, 7, 2, 5, 8, 4, 6, 0, 1])` should return 9 (the longest consecutive sequence is [0, 1, 2, 3, 4, 5, 6, 7, 8]) - `longest_consecutive([])` should return 0

Approach: We'll use a hash set for $O(1)$ lookups. For each number, we check if it's the start of a sequence (i.e., `num-1` is not in the set), then count upward as long as consecutive numbers exist.

```
def longest_consecutive(nums: list[int]) -> int:
    if not nums:
        return 0

    # Create a set for O(1) lookups
    num_set = set(nums)
    max_length = 0

    for num in num_set:
        # Only process numbers that are the start of a sequence
        if num - 1 not in num_set:
            current_num = num
            current_length = 1

            # Count consecutive numbers
            while current_num + 1 in num_set:
                current_num += 1
                current_length += 1

            max_length = max(max_length, current_length)

    return max_length
```

Visual Example:

```
nums = [100, 4, 200, 1, 3, 2]
num_set = {1, 2, 3, 4, 100, 200}
```

For `num = 1`:

```
1-1 = 0 is not in num_set, so 1 is the start of a sequence
current_length = 1
Check 1+1 = 2: Yes, in num_set
current_length = 2
Check 2+1 = 3: Yes, in num_set
current_length = 3
```

```
Check 3+1 = 4: Yes, in num_set
    current_length = 4
Check 4+1 = 5: No, not in num_set
max_length = 4
```

For num = 2, 3, 4:
 Not the start of a sequence (previous number exists in set)

For num = 100:
 100-1 = 99 is not in num_set, so 100 is the start of a sequence
 current_length = 1
 Check 100+1 = 101: No, not in num_set
 max_length = max(4, 1) = 4

For num = 200:
 200-1 = 199 is not in num_set, so 200 is the start of a sequence
 current_length = 1
 Check 200+1 = 201: No, not in num_set
 max_length = max(4, 1) = 4

Result: 4

Time Complexity: $O(n)$ - Although we have nested loops, each number is only considered once as the start of a sequence
Space Complexity: $O(n)$ - We store all numbers in a set

Two Pointers

Theory

The Two Pointers technique uses two pointers to traverse an array or string. These pointers can move toward each other, in the same direction at different speeds, or even start at different positions.

Common Two Pointer Patterns:

1. **Opposite Direction:** Pointers start from opposite ends and move toward each other
 - Used for: Reversing arrays, two sum on sorted arrays, container with most water
 - Time Complexity: Usually $O(n)$
2. **Same Direction:** Both pointers move in the same direction
 - Used for: Removing duplicates, finding specific patterns, merge operations
 - Time Complexity: Usually $O(n)$
3. **Fast and Slow:** One pointer moves faster than the other
 - Used for: Cycle detection, finding middle elements
 - Time Complexity: Usually $O(n)$

Visual Explanation:

Opposite Direction (Two Sum II):

Sorted Array: [1, 2, 4, 6, 8, 9, 10], Target = 10

```
left                                right
↓                                  ↓
[1, 2, 4, 6, 8, 9, 10]
```

$1 + 10 = 11 > 10$, so move right pointer left

```
left                                right
↓                                  ↓
[1, 2, 4, 6, 8, 9, 10]
```

$1 + 9 = 10 == 10$, found a match!

Same Direction (Remove Duplicates):

Array: [0, 0, 1, 1, 1, 2, 2, 3, 3, 4]

```
slow  fast
↓     ↓
[0, 0, 1, 1, 1, 2, 2, 3, 3, 4]
```

`nums[slow] = nums[fast]`, so just move fast

```
slow    fast
↓       ↓
[0, 0, 1, 1, 1, 2, 2, 3, 3, 4]
```

`nums[slow] != nums[fast]`, so update slow+1 with fast value

```
slow    fast
↓       ↓
[0, 1, 1, 1, 1, 2, 2, 3, 3, 4]
```

Fast and Slow (Linked List Cycle Detection):

Linked List with Cycle:

```
head→ 1 → 2 → 3 → 4
           ↑
```

Initially:

```
slow = head, fast = head
```

After iterations:

slow moves one step at a time

fast moves two steps at a time

Eventually:

slow = 3, fast = 3 (they meet) -> Cycle detected!

When to Use Two Pointers Use Two Pointers when: - The problem involves sorted arrays or specific positioning - You need to find pairs or triplets with certain properties - You need to compare elements from different positions - The problem involves finding a middle point, cycle, or palindrome - You need to process array elements in relation to each other

Example Problems

Easy: Valid Palindrome Problem: Given a string `s`, write a function that returns true if it is a palindrome, considering only alphanumeric characters and ignoring case. A palindrome reads the same forward and backward.

Examples: - `is_palindrome("A man, a plan, a canal: Panama")` should return True - `is_palindrome("race a car")` should return False - `is_palindrome(" ")` should return True (a single space contains no alphanumeric characters) - `is_palindrome("OP")` should return False

Approach: Use two pointers starting from the ends of the string and moving toward the center, skipping non-alphanumeric characters.

```
def is_palindrome(s: str) -> bool:
    # Convert to lowercase and initialize pointers
    s = s.lower()
    left, right = 0, len(s) - 1

    while left < right:
        # Skip non-alphanumeric characters
        if not s[left].isalnum():
            left += 1
            continue
        if not s[right].isalnum():
            right -= 1
            continue

        # Check if characters match
        if s[left] != s[right]:
            return False
```

```

    left += 1
    right -= 1

    return True

```

Visual Example:

```

s = "A man, a plan, a canal: Panama"
s (lowercase) = "a man, a plan, a canal: panama"

```

```

left = 0, right = 29
s[left] = 'a', s[right] = 'a' -> Match, move pointers
left = 1, right = 28
s[left] = ' ' -> Skip non-alphanumeric
left = 2, right = 28
s[left] = 'm', s[right] = 'm' -> Match, move pointers
...
(continue checking pairs of characters)
...
Eventually left >= right, all characters matched, return True

```

Time Complexity: $O(n)$ - Each character is visited at most once Space Complexity: $O(1)$ - We only use a constant amount of extra space

Medium: 3Sum Problem: Given an array of integers `nums`, write a function that returns all triplets `[nums[i], nums[j], nums[k]]` such that $i \neq j$, $i \neq k$, $j \neq k$, and $nums[i] + nums[j] + nums[k] == 0$. The solution set must not contain duplicate triplets.

Examples: - `three_sum([-1, 0, 1, 2, -1, -4])` should return `[[-1, -1, 2], [-1, 0, 1]]` - `three_sum([0, 1, 1])` should return `[]` - `three_sum([0, 0, 0])` should return `[[0, 0, 0]]`

Approach: Sort the array first, then for each element, use two pointers to find the other two elements that sum to zero.

```

def three_sum(nums: list[int]) -> list[list[int]]:
    result = []
    nums.sort()

    for i in range(len(nums) - 2):
        # Skip duplicates
        if i > 0 and nums[i] == nums[i - 1]:
            continue

        # Use two pointers for the remaining array
        left, right = i + 1, len(nums) - 1

```

```

while left < right:
    total = nums[i] + nums[left] + nums[right]

    if total < 0:
        left += 1
    elif total > 0:
        right -= 1
    else:
        result.append([nums[i], nums[left], nums[right]])

        # Skip duplicates
        while left < right and nums[left] == nums[left + 1]:
            left += 1
        while left < right and nums[right] == nums[right - 1]:
            right -= 1

        left += 1
        right -= 1

return result

```

Visual Example:

nums = [-1, 0, 1, 2, -1, -4]
sorted: [-4, -1, -1, 0, 1, 2]

```

i=0, nums[i]=-4:
    left=1, right=5: -4 + -1 + 2 = -3 < 0, move left
    left=2, right=5: -4 + -1 + 2 = -3 < 0, move left
    left=3, right=5: -4 + 0 + 2 = -2 < 0, move left
    left=4, right=5: -4 + 1 + 2 = -1 < 0, move left
    left=5, right=5: left >= right, move to next i

i=1, nums[i]=-1:
    left=2, right=5: -1 + -1 + 2 = 0, add [-1, -1, 2] to result
    left=3, right=4: -1 + 0 + 1 = 0, add [-1, 0, 1] to result
    left=4, right=3: left >= right, move to next i

i=2, nums[i]=-1: (skip as duplicate)

i=3, nums[i]=0:
    left=4, right=5: 0 + 1 + 2 = 3 > 0, move right
    left=4, right=4: left >= right, move to next i

```

Result: [[-1, -1, 2], [-1, 0, 1]]

Time Complexity: $O(n^2)$ - Sorting is $O(n \log n)$, and the two-pointer approach

is $O(n^2)$ Space Complexity: $O(1)$ - Excluding the output array

Medium: Container With Most Water Problem: Given n non-negative integers `height` where each represents a point at coordinate $(i, \text{height}[i])$, write a function to find the maximum amount of water a container can store. The container is formed by selecting two heights and the area is calculated as the product of the width (distance between indices) and the height (minimum of the two heights).

Examples: - `max_area([1, 8, 6, 2, 5, 4, 8, 3, 7])` should return 49 (container between heights 8 at index 1 and height 7 at index 8) - `max_area([1, 1])` should return 1 - `max_area([4, 3, 2, 1, 4])` should return 16

Approach: Use two pointers at the beginning and end of the array. Calculate the area, then move the pointer with the smaller height inward.

```
def max_area(height: list[int]) -> int:
    left, right = 0, len(height) - 1
    max_water = 0

    while left < right:
        # Calculate width and height
        width = right - left
        h = min(height[left], height[right])

        # Update max area
        max_water = max(max_water, width * h)

        # Move the pointer with smaller height
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1

    return max_water
```

****Visual Example:****

`height = [1, 8, 6, 2, 5, 4, 8, 3, 7]`

left = 0, right = 8: width = 8, h = min(1, 7) = 1, area = 8 height[left] < height[right], move left

left = 1, right = 8: width = 7, h = min(8, 7) = 7, area = 49 height[left] > height[right], move right

left = 1, right = 7: width = 6, h = min(8, 3) = 3, area = 18 height[left] > height[right], move right

...and so on...

Maximum area = 49

Time Complexity: $O(n)$ - We process each element at most once

Space Complexity: $O(1)$ - We only use a constant amount of extra space

Sliding Window

Theory

The Sliding Window technique is a variation of the Two Pointers approach that maintains a "window" of elements in the array.

Common Sliding Window Patterns:

1. ****Fixed-Size Window****: Window size remains constant

- Used for: Finding maximum/minimum sum or average in a fixed-size subarray
- Time Complexity: $O(n)$

2. ****Variable-Size Window****: Window expands or contracts based on conditions

- Used for: Longest/shortest substring with certain properties
- Time Complexity: Usually $O(n)$

3. ****Dynamic Sliding Window with Hash Map/Counter****: Maintains a frequency count within the window

- Used for: Substring problems involving character frequency
- Time Complexity: $O(n)$

Visual Explanation:

Fixed-Size Window (Max Sum Subarray of Size K):

Array: [2, 1, 5, 1, 3, 2], $k = 3$

Window 1: [2, 1, 5], 1, 3, 2 $\uparrow \uparrow$ left right Sum = 8

Window 2: 2, [1, 5, 1], 3, 2 $\uparrow \uparrow$ left right Sum = 7

Window 3: 2, 1, [5, 1, 3], 2 $\uparrow \uparrow$ left right Sum = 9

Window 4: 2, 1, 5, [1, 3, 2] $\uparrow \uparrow$ left right Sum = 6

Max Sum = 9

Variable-Size Window (Longest Substring Without Repeating Characters):

String: "abcabcbb"

Initial: Window = "" $a \rightarrow$ Window = "a" $b \rightarrow$ Window = "ab" $c \rightarrow$ Window

= "abc" a→ Found duplicate 'a', shrink window from left until 'a' is removed
 Window = "bca" b→ Found duplicate 'b', shrink window from left until 'b' is removed
 Window = "cab" c→ Found duplicate 'c', shrink window from left until 'c' is removed
 Window = "abc" b→ Found duplicate 'b', shrink window from left until 'b' is removed
 Window = "cb"

Longest = 3 characters

When to Use Sliding Window

Use Sliding Window when:

- The problem involves subarrays or substrings
- You need to find a continuous range with specific properties
- The problem can be solved by expanding/contracting a range
- You're calculating running sums, averages, or other aggregate metrics
- You need to find longest/shortest/optimal subarray with a specific property

Example Problems

Easy: Maximum Sum Subarray of Size K

****Problem**:** Given an array of integers `nums` and an integer `k`, write a function that returns the maximum sum of a subarray of size k.

****Examples**:**

- `max\_sum\_subarray([2, 1, 5, 1, 3, 2], 3)` should return `9` (subarray [5, 1, 3])
- `max\_sum\_subarray([1, 4, 2, 10, 2, 3, 1, 0, 20], 4)` should return `24` (subarray [2, 10, 2, 3])
- `max\_sum\_subarray([1, 1, 1, 1, 1], 2)` should return `2` (any subarray of size 2)
- `max\_sum\_subarray([1, 2], 3)` should return `-1` (no subarray of size 3 exists)

****Approach**:** Use a sliding window of size k to track the sum of the current window.

```
```python
def max_sum_subarray(nums: list[int], k: int) -> int:
 # Check if k is valid
 if k > len(nums):
 return -1

 # Calculate sum of first window
 current_sum = sum(nums[:k])
 max_sum = current_sum

 # Slide the window
 for i in range(k, len(nums)):
 # Remove the element going out and add the element coming in
 current_sum = current_sum - nums[i - k] + nums[i]
 max_sum = max(max_sum, current_sum)
```

```
 return max_sum
```

### Visual Example:

```
nums = [2, 1, 5, 1, 3, 2], k = 3
```

```
Initial window: [2, 1, 5]
current_sum = 8, max_sum = 8
```

```
Slide window:
 Remove 2, add 1: [1, 5, 1]
 current_sum = 7, max_sum = 8
```

```
Slide window:
 Remove 1, add 3: [5, 1, 3]
 current_sum = 9, max_sum = 9
```

```
Slide window:
 Remove 5, add 2: [1, 3, 2]
 current_sum = 6, max_sum = 9
```

Result: 9

Time Complexity:  $O(n)$  - We process each element once Space Complexity:  $O(1)$   
- We only use a constant amount of extra space

### Medium: Longest Substring Without Repeating Characters Problem:

Given a string `s`, write a function that returns the length of the longest substring without repeating characters.

**Examples:** - `length_of_longest_substring("abcabcbb")` should return 3 (the substring "abc") - `length_of_longest_substring("bbbbbb")` should return 1 (the substring "b") - `length_of_longest_substring("pwwkew")` should return 3 (the substring "wke") - `length_of_longest_substring("")` should return 0

**Approach:** Use a sliding window with a hash map to track character positions.

```
def length_of_longest_substring(s: str) -> int:
 char_index = {} # Character -> Last seen index
 max_length = 0
 start = 0

 for end, char in enumerate(s):
 # If character is already in the window, shrink window from the left
 if char in char_index and char_index[char] >= start:
 start = char_index[char] + 1
```

```

else:
 # Update max length
 max_length = max(max_length, end - start + 1)

 # Update character position
 char_index[char] = end

return max_length

```

#### Visual Example:

```
s = "abcabcbb"
```

Initialize: start = 0, max\_length = 0, char\_index = {}

Iteration 1: end = 0, char = 'a'  
 'a' not in char\_index, update max\_length = 1  
 char\_index = {'a': 0}

Iteration 2: end = 1, char = 'b'  
 'b' not in char\_index, update max\_length = 2  
 char\_index = {'a': 0, 'b': 1}

Iteration 3: end = 2, char = 'c'  
 'c' not in char\_index, update max\_length = 3  
 char\_index = {'a': 0, 'b': 1, 'c': 2}

Iteration 4: end = 3, char = 'a'  
 'a' in char\_index at position 0, move start to 1  
 char\_index = {'a': 3, 'b': 1, 'c': 2}  
 Window is "bca", max\_length = 3

...and so on...

Result: 3

Time Complexity:  $O(n)$  - We process each character once  
 Space Complexity:  $O(\min(m, n))$  - Where  $m$  is the size of the character set, bounded by the string length  $n$

**Medium: Minimum Size Subarray Sum Problem:** Given an array of positive integers `nums` and a positive integer `target`, write a function that returns the minimal length of a contiguous subarray whose sum is greater than or equal to the `target`. If there is no such subarray, return 0.

**Examples:** - `min_subarray_len([2, 3, 1, 2, 4, 3], 7)` should return 2 (the subarray [4, 3]) - `min_subarray_len([1, 4, 4], 4)` should return 1 (the

subarray [4]) - min\_subarray\_len([1, 1, 1, 1, 1, 1, 1, 1], 11) should return 0 (no solution)

**Approach:** Use a variable-size sliding window that expands until the sum is target, then contracts from the left to minimize length.

```
def min_subarray_len(nums: list[int], target: int) -> int:
 left = 0
 current_sum = 0
 min_length = float('inf')

 for right in range(len(nums)):
 # Expand window by adding right element
 current_sum += nums[right]

 # Contract window from the left until sum < target
 while current_sum >= target:
 # Update minimum length
 min_length = min(min_length, right - left + 1)

 # Remove leftmost element from sum
 current_sum -= nums[left]
 left += 1

 return min_length if min_length != float('inf') else 0
```

**Visual Example:**

nums = [2, 3, 1, 2, 4, 3], target = 7

Initial: left = 0, right = 0, current\_sum = 0, min\_length = inf

Iteration 1: right = 0, add nums[0] = 2  
current\_sum = 2 < target, continue

Iteration 2: right = 1, add nums[1] = 3  
current\_sum = 5 < target, continue

Iteration 3: right = 2, add nums[2] = 1  
current\_sum = 6 < target, continue

Iteration 4: right = 3, add nums[3] = 2  
current\_sum = 8 >= target, can shrink  
min\_length = 4  
remove nums[0] = 2, current\_sum = 6 < target, stop shrinking

Iteration 5: right = 4, add nums[4] = 4  
current\_sum = 10 >= target, can shrink

```

min_length = min(4, 3) = 3
remove nums[1] = 3, current_sum = 7 = target, can still shrink
min_length = min(3, 2) = 2
remove nums[2] = 1, current_sum = 6 < target, stop shrinking

```

```

Iteration 6: right = 5, add nums[5] = 3
current_sum = 9 >= target, can shrink
min_length = min(2, 3) = 2
remove nums[3] = 2, current_sum = 7 = target, can still shrink
min_length = min(2, 2) = 2
remove nums[4] = 4, current_sum = 3 < target, stop shrinking

```

Result: 2

Time Complexity:  $O(n)$  - Each element is processed at most twice (once added, once removed) Space Complexity:  $O(1)$  - We only use a constant amount of extra space

## Stack and Queue

### Theory

Stacks and queues are fundamental data structures that follow specific principles for adding and removing elements.

**Stack** A stack follows the Last-In-First-Out (LIFO) principle, where the last element added is the first one removed.

**Key Operations and Complexity:** - Push (add to top):  $O(1)$  - Pop (remove from top):  $O(1)$  - Peek (view top element):  $O(1)$

### Stack Visualization:

```

5 ← Top (Last In, First Out)
8
3
1
7

```

### Python Implementation:

```

Using a list (built-in)
stack = []
stack.append(1) # Push
stack.append(2)
top = stack.pop() # Pop (returns 2)

Using collections.deque (more efficient for large stacks)

```

```

from collections import deque
stack = deque()
stack.append(1) # Push
stack.append(2)
top = stack.pop() # Pop (returns 2)

```

**Queue** A queue follows the First-In-First-Out (FIFO) principle, where the first element added is the first one removed.

**Key Operations and Complexity:** - Enqueue (add to back):  $O(1)$  - Dequeue (remove from front):  $O(1)$  - Peek (view front element):  $O(1)$

**Queue Visualization:**

```

Front Back
↓ ↓
7 → 1 → 3 → 8
(First In, First Out)

```

**Python Implementation:**

```

Using collections.deque (efficient)
from collections import deque
queue = deque()
queue.append(1) # Enqueue
queue.append(2)
front = queue.popleft() # Dequeue (returns 1)

Using queue.Queue (thread-safe)
from queue import Queue
q = Queue()
q.put(1) # Enqueue
q.put(2)
front = q.get() # Dequeue (returns 1)

```

**Monotonic Stack/Queue** A monotonic stack/queue maintains elements in strictly increasing or decreasing order.

**Monotonic Stack Visualization (Increasing):**

```

9 ← Top
7
5
3
1

```

**Monotonic Stack Applications:** - Next/previous greater/smaller element problems - Largest rectangle in histogram - Maximum area under skyline

**When to Use Stacks and Queues** Use stacks when: - You need to process elements in reverse order - You're implementing a recursive algorithm iteratively - You need to track function calls (call stack) - You're parsing expressions (e.g., parentheses matching) - You need to implement undo/redo functionality

Use queues when: - You need to process elements in order of arrival - You're implementing breadth-first search - You're managing tasks in the order they were received - You need a buffer

### Example Problems

**Easy: Valid Parentheses Problem:** Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, write a function that determines if the input string is valid. An input string is valid if: 1. Open brackets must be closed by the same type of brackets. 2. Open brackets must be closed in the correct order. 3. Every close bracket has a corresponding open bracket of the same type.

**Examples:** - `is_valid("()")` should return `True` - `is_valid("() [] {}")` should return `True` - `is_valid("(")` should return `False` - `is_valid("([])")` should return `False` - `is_valid("{[]}")` should return `True`

**Approach:** Use a stack to keep track of opening brackets. When we encounter a closing bracket, check if it matches the most recent opening bracket.

```
def is_valid(s: str) -> bool:
 stack = []
 bracket_map = {')': '(', '}': '{', ']': '['}

 for char in s:
 # If it's an opening bracket, push to stack
 if char not in bracket_map:
 stack.append(char)
 # If it's a closing bracket, check if it matches the top of the stack
 else:
 if not stack or stack.pop() != bracket_map[char]:
 return False

 # Stack should be empty if all brackets are matched
 return len(stack) == 0
```

### Visual Example:

`s = "{[]}"`

Iteration 1: `char = '{'`  
It's an opening bracket, push to stack  
`stack = ['{']`

Iteration 2: `char = '['`

```
It's an opening bracket, push to stack
stack = ['{', '[']
```

```
Iteration 3: char = ']'
It's a closing bracket, check if it matches top of stack
Pop '[' from stack, matches ']', continue
stack = ['{']
```

```
Iteration 4: char = '}'
It's a closing bracket, check if it matches top of stack
Pop '{' from stack, matches '}', continue
stack = []
```

Stack is empty, return True

Time Complexity:  $O(n)$  - We scan the string once Space Complexity:  $O(n)$  - In the worst case, the stack could contain all opening brackets

**Medium: Daily Temperatures Problem:** Given an array of integers `temperatures` representing daily temperatures, write a function that returns an array `result` such that `result[i]` is the number of days you would have to wait after the *i*th day to get a warmer temperature. If there is no future day with a warmer temperature, put 0 in the result array.

**Examples:** - `daily_temperatures([73, 74, 75, 71, 69, 72, 76, 73])` should return `[1, 1, 4, 2, 1, 1, 0, 0]` - `daily_temperatures([30, 40, 50, 60])` should return `[1, 1, 1, 0]` - `daily_temperatures([30, 60, 90])` should return `[1, 1, 0]`

**Approach:** Use a monotonic decreasing stack to keep track of temperatures. When we find a higher temperature, pop elements from the stack and calculate the difference in indices.

```
def daily_temperatures(temperatures: list[int]) -> list[int]:
 n = len(temperatures)
 result = [0] * n
 stack = [] # Stack of indices

 for i in range(n):
 # While current temperature is higher than temperature at stack top
 while stack and temperatures[i] > temperatures[stack[-1]]:
 prev_idx = stack.pop()
 result[prev_idx] = i - prev_idx

 stack.append(i)

 return result
```

### Visual Example:

```
temperatures = [73, 74, 75, 71, 69, 72, 76, 73]
result = [0, 0, 0, 0, 0, 0, 0, 0]
```

```
Iteration 1: i = 0, temp = 73
 Stack empty, push index 0
 stack = [0]
```

```
Iteration 2: i = 1, temp = 74
 74 > temperatures[0] (73), pop 0 from stack
 result[0] = 1 - 0 = 1
 stack = []
 Push index 1
 stack = [1]
```

```
Iteration 3: i = 2, temp = 75
 75 > temperatures[1] (74), pop 1 from stack
 result[1] = 2 - 1 = 1
 stack = []
 Push index 2
 stack = [2]
```

```
Iteration 4: i = 3, temp = 71
 71 < temperatures[2] (75), push index 3
 stack = [2, 3]
```

```
Iteration 5: i = 4, temp = 69
 69 < temperatures[3] (71), push index 4
 stack = [2, 3, 4]
```

```
Iteration 6: i = 5, temp = 72
 72 > temperatures[4] (69), pop 4 from stack
 result[4] = 5 - 4 = 1
 72 > temperatures[3] (71), pop 3 from stack
 result[3] = 5 - 3 = 2
 72 < temperatures[2] (75), push index 5
 stack = [2, 5]
```

```
Iteration 7: i = 6, temp = 76
 76 > temperatures[5] (72), pop 5 from stack
 result[5] = 6 - 5 = 1
 76 > temperatures[2] (75), pop 2 from stack
 result[2] = 6 - 2 = 4
 stack = []
 Push index 6
```

```
stack = [6]
```

```
Iteration 8: i = 7, temp = 73
73 < temperatures[6] (76), push index 7
stack = [6, 7]
```

```
Final result = [1, 1, 4, 2, 1, 1, 0, 0]
```

Time Complexity:  $O(n)$  - Each index is pushed and popped at most once

Space Complexity:  $O(n)$  - In the worst case, the stack could contain all indices

**Medium: Implement Queue using Stacks** **Problem:** Implement a first-in-first-out (FIFO) queue using only two stacks. The implemented queue should support the standard queue operations: `push`, `peek`, `pop`, and `empty`.

**Examples:**

```
MyQueue queue = new MyQueue();
queue.push(1); // queue is: [1]
queue.push(2); // queue is: [1, 2]
queue.peek(); // return 1
queue.pop(); // return 1, queue is [2]
queue.empty(); // return false
```

**Approach:** Use one stack for pushing elements and another stack for popping elements. Transfer elements between stacks when necessary.

```
class MyQueue:
 def __init__(self):
 self.stack_push = [] # For push operations
 self.stack_pop = [] # For pop/peek operations

 def push(self, x: int) -> None:
 # Always push to the push stack
 self.stack_push.append(x)

 def _transfer(self) -> None:
 # If pop stack is empty, transfer all elements from push stack
 if not self.stack_pop:
 while self.stack_push:
 self.stack_pop.append(self.stack_push.pop())

 def pop(self) -> int:
 self._transfer()
 if self.stack_pop:
 return self.stack_pop.pop()
 return None
```

```

def peek(self) -> int:
 self._transfer()
 if self.stack_pop:
 return self.stack_pop[-1]
 return None

def empty(self) -> bool:
 return len(self.stack_push) == 0 and len(self.stack_pop) == 0

```

### Visual Example:

Operations:

- push(1)  
stack\_push = [1], stack\_pop = []
- push(2)  
stack\_push = [1, 2], stack\_pop = []
- peek()  
stack\_pop empty, transfer:  
stack\_push = [], stack\_pop = [2, 1]  
Return stack\_pop[-1] = 1
- pop()  
stack\_pop = [2, 1]  
Return stack\_pop.pop() = 1  
stack\_push = [], stack\_pop = [2]
- empty()  
stack\_push and stack\_pop not both empty  
Return False

Time Complexity: - push:  $O(1)$  - Direct push to a stack - pop: Amortized  $O(1)$   
 - Each element is transferred at most once - peek: Amortized  $O(1)$  - Same as pop - empty:  $O(1)$  - Simple length check Space Complexity:  $O(n)$  - For storing all elements across both stacks

Binary Search Theory Binary search is a divide-and-conquer algorithm that efficiently finds an element in a sorted array by repeatedly dividing the search space in half.

Key Principles:

The array must be sorted In each step, compare the target with the middle element If target equals middle element, return the index If target is less than middle element, search the left half If target is greater than middle element, search the right half Visual Explanation:

Sorted Array: [1, 3, 5, 7, 9, 11, 13, 15, 17, 19] Target: 9

Step 1: Calculate  $\text{mid} = (\text{left} + \text{right}) / 2 = (0 + 9) / 2 = 4$  Check  $\text{nums}[4] = 9 == \text{target}$ ? Yes, return 4.

Another example with  $\text{target} = 13$ :

Iteration 1:  $[1, 3, 5, 7, 9, 11, 13, 15, 17, 19] \uparrow \text{mid} = 4$   $\text{nums}[4] = 9 < 13$ , search right half

Iteration 2:  $[11, 13, 15, 17, 19] \uparrow \text{mid} = 7$   $\text{nums}[7] = 15 > 13$ , search left half

Iteration 3:  $[11, 13] \uparrow \text{mid} = 6$   $\text{nums}[6] = 13 == 13$ , found at index 6! Basic Implementation:

```
python def binary_search(nums: list[int], target: int) -> int:
 left, right = 0, len(nums) - 1
 while left <= right:
 mid = (left + right) // 2 # Or mid = left + (right - left) // 2 to avoid overflow
 if nums[mid] == target:
 return mid
 elif nums[mid] < target:
 left = mid + 1
 else:
 right = mid - 1
 return -1 # Target not found
```

Common Variations:

Finding the leftmost occurrence: `python def binary_search_leftmost(nums: list[int], target: int) -> int:` `left, right = 0, len(nums) - 1` `result = -1`

```
while left <= right:
 mid = (left + right) // 2
 if nums[mid] == target:
 result = mid # Found a match, but continue searching left
 right = mid - 1
 elif nums[mid] < target:
 left = mid + 1
 else:
 right = mid - 1
return result
```

3. **Binary search on answer space:** Sometimes binary search can be applied to problems that don't involve arrays but have a range of possible answers.

**When to Use Binary Search** Use binary search when: - The input array is sorted or can be sorted - You need to find an element in a sorted array - You need to find an insertion point or boundary - You need to find a value that satisfies a specific condition - The problem has a monotonic property (increasing or decreasing)

### Example Problems

**Easy: Binary Search Problem:** Given a sorted array of integers `nums` and an integer `target`, write a function that returns the index of `target` if it exists in `nums`, or -1 if it does not exist.

**Examples:** - `search([1, 2, 3, 4, 5], 3)` should return 2 - `search([-1, 0, 3, 5, 9, 12], 9)` should return 4 - `search([-1, 0, 3, 5, 9, 12], 2)` should return -1 - `search([], 0)` should return -1

**Approach:** Use the standard binary search algorithm on the sorted array.

```
def search(nums: list[int], target: int) -> int:
 left, right = 0, len(nums) - 1

 while left <= right:
 mid = left + (right - left) // 2

 if nums[mid] == target:
 return mid
 elif nums[mid] < target:
 left = mid + 1
 else:
 right = mid - 1

 return -1
```

Visual Example:

```
nums = [-1, 0, 3, 5, 9, 12], target = 9
```

Iteration 1:

```
left = 0, right = 5, mid = 2
nums[mid] = 3 < 9, search right half
left = 3, right = 5
```

Iteration 2:

```
left = 3, right = 5, mid = 4
nums[mid] = 9 == 9, found at index 4!
```

Time Complexity:  $O(\log n)$  - The search space is halved in each step

Space Complexity:  $O(1)$  - We only use a constant amount of extra space

Medium: Search in Rotated Sorted Array

Problem: Given a rotated sorted array `nums` and an integer `target`, write a function that returns the index of `target` if it exists in `nums`, otherwise return `-1`.

Examples:

```
search_rotated([4, 5, 6, 7, 0, 1, 2], 0) should return 4
search_rotated([4, 5, 6, 7, 0, 1, 2], 3) should return -1
search_rotated([1], 0) should return -1
search_rotated([1, 3], 3) should return 1
```

Approach: We need to modify the binary search to handle the rotation. First, find which half is sorted.

python

```
def search_rotated(nums: list[int], target: int) -> int:
 left, right = 0, len(nums) - 1

 while left <= right:
 mid = left + (right - left) // 2

 if nums[mid] == target:
 return mid

 # Check if left half is sorted
 if nums[left] <= nums[mid]:
 # Check if target is in the left sorted half
 if nums[left] <= target < nums[mid]:
 right = mid - 1 # Target is in the left sorted half
 else:
 left = mid + 1 # Target is in the right half
 # Right half is sorted
 else:
 # Check if target is in the right sorted half
 if nums[mid] < target <= nums[right]:
 left = mid + 1 # Target is in the right sorted half
 else:
 right = mid - 1 # Target is in the left half

 return -1
```

Visual Example:

```
nums = [4, 5, 6, 7, 0, 1, 2], target = 0
```

Iteration 1:

```
left = 0, right = 6, mid = 3
nums[mid] = 7 != 0
nums[left] = 4 <= nums[mid] = 7, left half is sorted
target = 0 < nums[left] = 4, target not in left half
```

```
left = mid + 1 = 4, right = 6
```

Iteration 2:

```
left = 4, right = 6, mid = 5
nums[mid] = 1 != 0
nums[left] = 0 <= nums[mid] = 1, left half is sorted
nums[left] = 0 <= target = 0 < nums[mid] = 1, target in left half
left = 4, right = mid - 1 = 4
```

Iteration 3:

```
left = 4, right = 4, mid = 4
nums[mid] = 0 == 0, found at index 4!
```

Time Complexity:  $O(\log n)$  - We're still performing binary search

Space Complexity:  $O(1)$  - We only use a constant amount of extra space

Medium: Find First and Last Position of Element in Sorted Array

Problem: Given a sorted array of integers nums and an integer target, write a function that

Examples:

search\_range([5, 7, 7, 8, 8, 10], 8) should return [3, 4]

search\_range([5, 7, 7, 8, 8, 10], 6) should return [-1, -1]

search\_range([], 0) should return [-1, -1]

search\_range([1], 1) should return [0, 0]

Approach: Use modified binary search twice - once to find the leftmost occurrence and once to

python

```
def search_range(nums: list[int], target: int) -> list[int]:
 # Helper function to find leftmost occurrence
 def find_leftmost():
 left, right = 0, len(nums) - 1
 result = -1

 while left <= right:
 mid = left + (right - left) // 2

 if nums[mid] == target:
 result = mid
 right = mid - 1 # Continue searching left
 elif nums[mid] < target:
 left = mid + 1
 else:
 right = mid - 1

 return result
```

```

Helper function to find rightmost occurrence
def find_rightmost():
 left, right = 0, len(nums) - 1
 result = -1

 while left <= right:
 mid = left + (right - left) // 2

 if nums[mid] == target:
 result = mid
 left = mid + 1 # Continue searching right
 elif nums[mid] < target:
 left = mid + 1
 else:
 right = mid - 1

 return result

left_idx = find_leftmost()
if left_idx == -1:
 return [-1, -1]

right_idx = find_rightmost()
return [left_idx, right_idx]
Visual Example:

```

```

nums = [5, 7, 7, 8, 8, 10], target = 8

```

Find leftmost:

Iteration 1:

```

left = 0, right = 5, mid = 2
nums[mid] = 7 < 8, search right half
left = 3, right = 5

```

Iteration 2:

```

left = 3, right = 5, mid = 4
nums[mid] = 8 == 8, result = 4, continue searching left
left = 3, right = 3

```

Iteration 3:

```

left = 3, right = 3, mid = 3
nums[mid] = 8 == 8, result = 3, continue searching left
left = 3, right = 2 (loop ends)
leftmost = 3

```

Find rightmost:

```

Iteration 1:
left = 0, right = 5, mid = 2
nums[mid] = 7 < 8, search right half
left = 3, right = 5

```

```

Iteration 2:
left = 3, right = 5, mid = 4
nums[mid] = 8 == 8, result = 4, continue searching right
left = 5, right = 5

```

```

Iteration 3:
left = 5, right = 5, mid = 5
nums[mid] = 10 > 8, search left half
left = 5, right = 4 (loop ends)
rightmost = 4

```

Result: [3, 4]

Time Complexity:  $O(\log n)$  - We perform binary search twice

Space Complexity:  $O(1)$  - We only use a constant amount of extra space

2. **\*\*Finding the rightmost occurrence:\*\***

```

```python
def binary_search_rightmost(nums: list[int], target: int) -> int:
    left, right = 0, len(nums) - 1
    result = -1

    while left <= right:
        mid = (left + right) // 2

        if nums[mid] == target:
            result = mid # Found a match, but continue searching right
            left = mid + 1
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return result

```

Linked Lists

Theory

A linked list is a linear data structure where elements are stored in nodes, and each node p

Types of Linked Lists:

1. ****Singly Linked List****: Each node has data and a reference to the next node
2. ****Doubly Linked List****: Each node has data and references to both next and previous nodes
3. ****Circular Linked List****: Last node points back to the first node

```
**Singly Linked List Visualization:**
```

1 • → 2 • → 3 • → 4 /
 Head Tail

```

**Doubly Linked List Visualization:**

```

	↑		↓		↑		↓		↑		↓
None←	/	1	•	→	•	2	•	→	•	3	/ → None
	Head		Tail								

```
**Circular Linked List Visualization:**
```

$$\bullet \rightarrow 2 \quad \bullet \rightarrow 3 \quad \bullet \xrightarrow{\downarrow} 4 \quad \bullet$$

Python Implementation:

```

python
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

```

Key Operations and Complexity:

- Access by index: $O(n)$
- Insert/Delete at beginning: $O(1)$
- Insert/Delete at end: $O(n)$ for singly linked list, $O(1)$ for doubly linked list with tail pointer
- Insert/Delete in the middle: $O(n)$ to find the position, $O(1)$ to perform the operation

Common Linked List Patterns:

1. **Two Pointers:** Fast and slow pointers, used for cycle detection, finding middle, etc.

2. **Dummy Head:** Creating a dummy head node to simplify operations on the list
3. **Reverse Linked List:** In-place reversal by changing links
4. **Merge Lists:** Combining multiple lists with specific criteria

When to Use Linked Lists Use linked lists when: - You need frequent insertions/deletions - You don't need random access - Memory allocation needs to be dynamic - You're implementing a queue, stack, or hash table (chaining)

Example Problems

Easy: Reverse Linked List Problem: Given the head of a singly linked list, write a function that reverses the list and returns the reversed list's head.

Examples: - `reverse_list(1->2->3->4->5)` should return `5->4->3->2->1` - `reverse_list(1->2)` should return `2->1` - `reverse_list(null)` should return `null`

Approach: Iteratively reverse the links of each node. Keep track of the previous, current, and next nodes.

```
def reverse_list(head: ListNode) -> ListNode:
    prev = None
    current = head

    while current:
        next_temp = current.next # Store next node
        current.next = prev      # Reverse the link
        prev = current           # Move prev forward
        current = next_temp      # Move current forward

    # After the loop, prev will point to the new head
    return prev
```

Visual Example:

Original: 1->2->3->4->5

Iteration 1:

```
current = 1, next_temp = 2
1->None, prev = 1, current = 2
```

Iteration 2:

```
current = 2, next_temp = 3
2->1->None, prev = 2, current = 3
```

Iteration 3:

```
current = 3, next_temp = 4
```

3->2->1->None, prev = 3, current = 4

Iteration 4:

current = 4, next_temp = 5
4->3->2->1->None, prev = 4, current = 5

Iteration 5:

current = 5, next_temp = None
5->4->3->2->1->None, prev = 5, current = None

Loop ends, return prev = 5

Result: 5->4->3->2->1

Time Complexity: $O(n)$ - We visit each node once Space Complexity: $O(1)$ - We only use a constant amount of extra space

Medium: Remove Nth Node From End of List Problem: Given the head of a linked list and an integer n , write a function that removes the n th node from the end of the list and returns its head.

Examples: - `remove_nth_from_end(1->2->3->4->5, 2)` should return `1->2->3->5` (remove 4th node) - `remove_nth_from_end(1, 1)` should return `null` (remove 1st node) - `remove_nth_from_end(1->2, 1)` should return `1` (remove 2nd node)

Approach: Use two pointers, with the first pointer n nodes ahead of the second. When the first reaches the end, the second will be at the node before the one to be deleted.

```
def remove_nth_from_end(head: ListNode, n: int) -> ListNode:
    # Create a dummy node to handle edge cases
    dummy = ListNode(0)
    dummy.next = head

    first = second = dummy

    # Advance first pointer by n+1 steps
    for _ in range(n + 1):
        if not first:
            break
        first = first.next

    # Move both pointers until first reaches the end
    while first:
        first = first.next
        second = second.next
```

```

    # Remove the nth node
    second.next = second.next.next

    return dummy.next

```

Visual Example:

head = 1->2->3->4->5, n = 2

Initialize:

```

dummy->1->2->3->4->5
first = dummy, second = dummy

```

Advance first n+1 steps:

```

dummy->1->2->3->4->5
      f           s

```

Move both pointers until first is null:

```

dummy->1->2->3->4->5
              f
              s

```

Remove node:

```

second.next = second.next.next
dummy->1->2->3->5

```

Return dummy.next = 1->2->3->5

Time Complexity: $O(n)$ - We traverse the list once Space Complexity: $O(1)$ - We only use a constant amount of extra space

Medium: Linked List Cycle II **Problem:** Given the head of a linked list, write a function that returns the node where the cycle begins. If there is no cycle, return null.

Examples: - `detect_cycle(1->2->3->4->2)` should return node with value 2 (where the cycle begins) - `detect_cycle(1->2->3->4)` should return null (no cycle) - `detect_cycle(1->1)` should return node with value 1 (cycle at the head)

Approach: Use Floyd's Tortoise and Hare algorithm (fast and slow pointers) to detect a cycle, then use a second set of pointers to find the start of the cycle.

```

def detect_cycle(head: ListNode) -> ListNode:
    if not head or not head.next:
        return None

```

```

    # Phase 1: Detect cycle using fast and slow pointers

```

```

slow = fast = head
has_cycle = False

while fast and fast.next:
    slow = slow.next
    fast = fast.next.next

    if slow == fast:
        has_cycle = True
        break

if not has_cycle:
    return None

# Phase 2: Find the start of the cycle
# Reset slow pointer to head, keep fast at meeting point
slow = head

while slow != fast:
    slow = slow.next
    fast = fast.next

return slow

```

Visual Example:

head = 1->2->3->4->2 (cycle from 4 back to 2)

Phase 1: Detect cycle

```

slow = 1, fast = 1
slow = 2, fast = 3
slow = 3, fast = 2
slow = 4, fast = 4 (meet)

```

Phase 2: Find cycle start

```

slow = 1, fast = 4
slow = 2, fast = 2 (meet at cycle start)

```

Return: Node with value 2

Time Complexity: $O(n)$ - In the worst case, we traverse the list twice

Space Complexity: $O(1)$ - We only use a constant amount of extra space

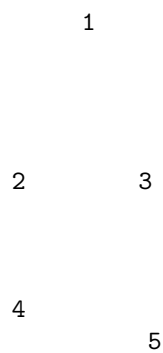
Trees

Theory

Trees are hierarchical data structures consisting of nodes connected by edges, with one node designated as the root.

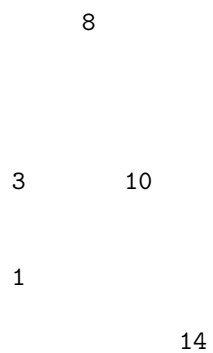
Types of Trees: **Binary Tree:** Each node has at most 2 children

Binary Tree Visualization:



Binary Search Tree (BST): Left child < node < right child

BST Visualization:



Balanced BST (AVL Tree): A self-balancing binary search tree

AVL Tree Visualization:



```

2          6

1          5

```

Basic Tree Node Definition:

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

```

Common Tree Properties:

- **Height:** The number of edges in the longest path from the root to a leaf
- **Depth:** The number of edges from the root to a specific node
- **Balance Factor:** The difference in height between left and right subtrees
- **Full Tree:** Every node has 0 or 2 children
- **Complete Tree:** All levels are filled except possibly the last

Tree Traversals:

1. **Pre-order:** Root $\rightarrow$ Left $\rightarrow$ Right

```

def preorder(root):
    if not root:
        return
    print(root.val) # Process root
    preorder(root.left) # Process left subtree
    preorder(root.right) # Process right subtree

```

2. **In-order:** Left $\rightarrow$ Root $\rightarrow$ Right

```

def inorder(root):
    if not root:
        return
    inorder(root.left) # Process left subtree
    print(root.val) # Process root
    inorder(root.right) # Process right subtree

```

3. **Post-order:** Left $\rightarrow$ Right $\rightarrow$ Root

```

def postorder(root):
    if not root:

```

```

        return
    postorder(root.left)  # Process left subtree
    postorder(root.right) # Process right subtree
    print(root.val)       # Process root

```

4. **Level-order** (BFS): Process nodes level by level

```

from collections import deque

def levelorder(root):
    if not root:
        return
    queue = deque([root])
    while queue:
        node = queue.popleft()
        print(node.val)  # Process node
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)

```

Traversal Visualization (for the first binary tree shown above): - Pre-order: 1, 2, 4, 3, 5 - In-order: 4, 2, 1, 3, 5 - Post-order: 4, 2, 5, 3, 1 - Level-order: 1, 2, 3, 4, 5

When to Use Trees Use trees when: - You need hierarchical data representation - You need efficient search, insert, and delete operations - You need to represent sorted data - You need to quickly find common ancestors or paths

Example Problems

Easy: Maximum Depth of Binary Tree Problem: Given the root of a binary tree, write a function that returns its maximum depth. The maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

Examples:

```

      3
     / \
    9  20
   /  \
  15   7

```

- `max_depth(above tree)` should return 3
- `max_depth(null)` should return 0
- `max_depth(1)` should return 1

Approach: Use recursion to find the depth of the left and right subtrees, then return the maximum plus one.

```
def max_depth(root: TreeNode) -> int:
    if not root:
        return 0

    left_depth = max_depth(root.left)
    right_depth = max_depth(root.right)

    return max(left_depth, right_depth) + 1
```

Visual Example:

```
max_depth(3):
    left_depth = max_depth(9):
        left_depth = max_depth(null) = 0
        right_depth = max_depth(null) = 0
        return max(0, 0) + 1 = 1

    right_depth = max_depth(20):
        left_depth = max_depth(15):
            left_depth = max_depth(null) = 0
            right_depth = max_depth(null) = 0
            return max(0, 0) + 1 = 1

        right_depth = max_depth(7):
            left_depth = max_depth(null) = 0
            right_depth = max_depth(null) = 0
            return max(0, 0) + 1 = 1

        return max(1, 1) + 1 = 2

    return max(1, 2) + 1 = 3
```

Time Complexity: $O(n)$ - We visit each node once Space Complexity: $O(h)$ - Where h is the height of the tree (for the recursion stack)

Medium: Validate Binary Search Tree Problem: Given the root of a binary tree, write a function that determines if it is a valid binary search tree (BST). A valid BST is defined as follows: - The left subtree of a node contains only nodes with keys less than the node's key. - The right subtree of a node contains only nodes with keys greater than the node's key. - Both the left and right subtrees must also be binary search trees.

Examples:

2

```

    / \
   1   3

```

- `is_valid_bst(above tree)` should return `True`

```

      5
     / \
    1   4
   / \
  3   6

```

- `is_valid_bst(above tree)` should return `False` (the value 3 is not greater than 5)

Approach: Use recursion with a range validation. Each node's value must be within a valid range based on its position in the tree.

```

def is_valid_bst(root: TreeNode) -> bool:
    def validate(node, low=float('-inf'), high=float('inf')):
        if not node:
            return True

        # Check if current node's value is within range
        if node.val <= low or node.val >= high:
            return False

        # Validate left subtree (must be less than current node)
        # Validate right subtree (must be greater than current node)
        return validate(node.left, low, node.val) and validate(node.right, node.val, high)

    return validate(root)

```

Visual Example:

```

is_valid_bst(5):
    validate(5, -inf, inf):
        node.val = 5, in range? Yes

        validate(1, -inf, 5):
            node.val = 1, in range? Yes
            validate(null, -inf, 1) = True
            validate(null, 1, 5) = True
            return True

        validate(4, 5, inf):
            node.val = 4, in range? No (4 is not > 5)
            return False

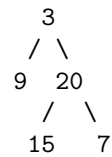
    return False

```

Time Complexity: $O(n)$ - We visit each node once Space Complexity: $O(h)$ - Where h is the height of the tree

Medium: Binary Tree Level Order Traversal Problem: Given the root of a binary tree, write a function that returns the level order traversal of its nodes' values (i.e., from left to right, level by level).

Examples:



- `level_order(above tree)` should return `[[3], [9, 20], [15, 7]]`
- `level_order(null)` should return `[]`
- `level_order(1)` should return `[[1]]`

Approach: Use BFS with a queue, keeping track of the current level.

```
from collections import deque

def level_order(root: TreeNode) -> list[list[int]]:
    if not root:
        return []

    result = []
    queue = deque([root])

    while queue:
        level_size = len(queue)
        level_nodes = []

        for _ in range(level_size):
            node = queue.popleft()
            level_nodes.append(node.val)

            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

        result.append(level_nodes)

    return result

**Visual Example:**
```

level_order(3): Initialize: result = [], queue = [3]

Level 1: level_size = 1, level_nodes = [] Process node 3: level_nodes = [3] Add children: queue = [9, 20] result = [[3]]

Level 2: level_size = 2, level_nodes = [] Process node 9: level_nodes = [9] No children Process node 20: level_nodes = [9, 20] Add children: queue = [15, 7] result = [[3], [9, 20]]

Level 3: level_size = 2, level_nodes = [] Process node 15: level_nodes = [15] No children Process node 7: level_nodes = [15, 7] No children result = [[3], [9, 20], [15, 7]]

Queue empty, return result = [[3], [9, 20], [15, 7]]

Time Complexity: $O(n)$ - We visit each node once

Space Complexity: $O(n)$ - In the worst case, the queue could contain all nodes at the bottom

Heaps/Priority Queues

Theory

A heap is a complete binary tree where every parent node has a value that satisfies a specific property. In a min-heap, each parent is less than or equal to its children. In a max-heap, each parent is greater than or equal to its children.

Key Properties:

- Complete binary tree structure
- Heap property (min-heap or max-heap)
- Root element is always the minimum (min-heap) or maximum (max-heap)
- Efficient insertion and extraction

****Min-Heap Visualization:****

1

2 3

4 5

****Max-Heap Visualization:****

10

7 9

5 6

Python Implementation:

Python's built-in `heapq` module implements a min-heap:

```
```python
import heapq

Create a heap
heap = []
heapq.heapify(heap) # Transform list into a heap (in-place)

Add elements
heapq.heappush(heap, 5)
heapq.heappush(heap, 3)
heapq.heappush(heap, 7)

Remove smallest element
smallest = heapq.heappop(heap) # Returns 3

Get smallest without removing
smallest = heap[0] # Peek at the smallest element

For a max-heap, negate the values
max_heap = []
heapq.heappush(max_heap, -5) # Add -5
max_largest = -heapq.heappop(max_heap) # Get the maximum value (5)
```
```

Array Representation of Heap: A heap can be represented as an array where for each element at index i : - Left child is at index $2i + 1$ - Right child is at index $2i + 2$ - Parent is at index $(i - 1) // 2$

Array Representation Visualization:

Min-Heap:

Array: [1, 2, 3, 4, 5]

1 (index 0)

2 3 (indices 1, 2)

4 5 (indices 3, 4)

Key Operations and Complexity:

- Build Heap: $O(n)$
- Insert: $O(\log n)$
- Extract Min/Max: $O(\log n)$
- Peek at Min/Max: $O(1)$
- Heapify (percolate down): $O(\log n)$

When to Use Heaps Use heaps when: - You need quick access to the minimum/maximum element - You need to efficiently get the k smallest/largest elements - You need to implement a priority queue - You're dealing with a data stream and need to maintain sorted behavior

Example Problems

Easy: Kth Largest Element in a Stream Problem: Design a class to find the kth largest element in a stream of numbers. The class should support the following operations: 1. `__init__(k, nums)`: Initialize the class with an integer k and an array of integers nums. 2. `add(val)`: Adds the integer val to the stream and returns the element representing the kth largest element in the stream.

Examples:

```
KthLargest kthLargest = new KthLargest(3, [4, 5, 8, 2]);
kthLargest.add(3);    // return 4
kthLargest.add(5);    // return 5
kthLargest.add(10);   // return 5
kthLargest.add(9);    // return 8
kthLargest.add(4);    // return 8
```

Approach: Maintain a min-heap of size k. When adding a new element, add it to the heap and remove the smallest if the heap size exceeds k.

```
import heapq

class KthLargest:
    def __init__(self, k: int, nums: list[int]):
        self.k = k
        self.heap = []
```

```

        # Initialize the heap with the first k elements
        for num in nums:
            self.add(num)

    def add(self, val: int) -> int:
        # Add the new value to the heap
        heapq.heappush(self.heap, val)

        # If heap size exceeds k, remove the smallest
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)

        # Return the kth largest (smallest in our min-heap)
        return self.heap[0]

```

Visual Example:

KthLargest(3, [4, 5, 8, 2]):

Initialize: k = 3, heap = []

Add 4: heap = [4]

Add 5: heap = [4, 5]

Add 8: heap = [4, 5, 8]

Add 2: heap = [2, 4, 5, 8]

Remove smallest: heap = [4, 5, 8]

kthLargest.add(3):

Add 3: heap = [3, 4, 5, 8]

Remove smallest: heap = [4, 5, 8]

Return 4

kthLargest.add(5):

Add 5: heap = [4, 5, 5, 8]

Remove smallest: heap = [5, 5, 8]

Return 5

... and so on

Time Complexity: - Initialization: $O(n \log k)$ - For adding n elements - Add operation: $O(\log k)$ - Heap operations with k elements Space Complexity: $O(k)$
 - We maintain a heap of size k

Medium: Top K Frequent Elements Problem: Given an integer array **nums** and an integer **k**, write a function that returns the **k** most frequent elements. The result can be in any order.

Examples: - `top_k_frequent([1, 1, 1, 2, 2, 3], 2)` should return `[1, 2]` - `top_k_frequent([1], 1)` should return `[1]` - `top_k_frequent([3, 0, 1, 0], 1)` should return `[0]`

Approach: Use a hash map to count frequencies, then use a heap to find the k most frequent elements.

```
from collections import Counter
import heapq

def top_k_frequent(nums: list[int], k: int) -> list[int]:
    # Count frequencies
    counter = Counter(nums)

    # Use a min-heap to keep track of the k most frequent elements
    heap = []

    for num, freq in counter.items():
        # Push to heap
        heapq.heappush(heap, (freq, num))

        # If heap size exceeds k, remove the least frequent
        if len(heap) > k:
            heapq.heappop(heap)

    # Extract numbers from heap (most frequent first)
    result = [num for _, num in heap]

    return result
```

Visual Example:

```
top_k_frequent([1, 1, 1, 2, 2, 3], 2):
    counter = {1: 3, 2: 2, 3: 1}
```

```
Process num=1, freq=3:
    Push (3, 1) to heap: heap = [(3, 1)]
```

```
Process num=2, freq=2:
    Push (2, 2) to heap: heap = [(2, 2), (3, 1)]
```

```
Process num=3, freq=1:
    Push (1, 3) to heap: heap = [(1, 3), (2, 2), (3, 1)]
    Heap size > k, remove (1, 3): heap = [(2, 2), (3, 1)]
```

```
Result = [2, 1]
```

Time Complexity: $O(n \log k)$ - Where n is the length of the input array Space

Complexity: $O(n)$ - For the counter and heap

Medium: Merge K Sorted Lists **Problem:** Given an array of k linked-lists `lists`, each linked-list is sorted in ascending order. Write a function that merges all the linked-lists into one sorted linked-list and returns it.

Examples: - `merge_k_lists([[1,4,5], [1,3,4], [2,6]])` should return `[1,1,2,3,4,4,5,6]` - `merge_k_lists([])` should return `[]` - `merge_k_lists([[]])` should return `[]`

Approach: Use a min-heap to efficiently merge the lists by always selecting the smallest element from all list heads.

```
import heapq

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def merge_k_lists(lists: list[ListNode]) -> ListNode:
    # Create a dummy head
    dummy = ListNode(0)
    current = dummy

    # Create a min heap
    # We need to use a counter as a tiebreaker since ListNode objects aren't comparable
    from itertools import count
    counter = count()
    heap = []

    # Add the first node from each list to the heap
    for head in lists:
        if head:
            # Store (value, counter, node) in the heap
            heapq.heappush(heap, (head.val, next(counter), head))

    # Process the heap
    while heap:
        # Get the smallest value
        val, _, node = heapq.heappop(heap)

        # Add it to our result list
        current.next = node
        current = current.next

        # If there are more nodes in this list, add the next one to the heap
```

```

        if node.next:
            heapq.heappush(heap, (node.next.val, next(counter), node.next))

    return dummy.next

```

Visual Example:

merge_k_lists([[1,4,5], [1,3,4], [2,6]]):

Initialize:

```

dummy -> null, current = dummy
heap = [(1, 0, list1.head), (1, 1, list2.head), (2, 2, list3.head)]

```

Iteration 1:

```

Pop (1, 0, list1.head) from heap
current.next = list1.head, current = list1.head
Push (4, 3, list1.head.next) to heap
heap = [(1, 1, list2.head), (2, 2, list3.head), (4, 3, list1.head.next)]

```

Iteration 2:

```

Pop (1, 1, list2.head) from heap
current.next = list2.head, current = list2.head
Push (3, 4, list2.head.next) to heap
heap = [(2, 2, list3.head), (3, 4, list2.head.next), (4, 3, list1.head.next)]

```

... and so on

Result: 1->1->2->3->4->4->5->6

Time Complexity: $O(N \log k)$ where N is the total number of nodes and k is the number of lists
 Space Complexity: $O(k)$ for the priority queue

Trie Data Structure

Theory

A Trie (pronounced “try”) is a tree-like data structure used to store a collection of strings. It’s particularly useful for efficient string operations like prefix lookups.

Key Properties:

1. **Structure:** Each node represents a character or a complete word
2. **Path:** Following a path from the root forms a string
3. **Shared Prefixes:** Strings with common prefixes share the same initial nodes
4. **End Markers:** Special markers indicate complete words

Trie Visualization:

Asterisks indicate the end of a complete word

```
class TrieNode:
    def __init__(self):
        self.children = {} # Map from character to TrieNode
        self.is_end_of_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end_of_word = True
```

```

def search(self, word: str) -> bool:
    node = self.root
    for char in word:
        if char not in node.children:
            return False
        node = node.children[char]
    return node.is_end_of_word

def starts_with(self, prefix: str) -> bool:
    node = self.root
    for char in prefix:
        if char not in node.children:
            return False
        node = node.children[char]
    return True

```

Key Operations and Complexity:

- Insert: $O(m)$ where m is the length of the word
- Search: $O(m)$ where m is the length of the word
- Prefix Search: $O(m)$ where m is the length of the prefix
- Space Complexity: $O(n*m)$ where n is the number of words and m is the average length

When to Use Tries Use Tries when: - You need efficient prefix searching
 - You're working with a dictionary or word list - You need to validate strings against a vocabulary - You're implementing autocomplete or spell checking - You need to find words with common prefixes

Example Problems

Easy: Implement Trie (Prefix Tree) Problem: Implement a trie with `insert`, `search`, and `startsWith` methods. The trie should support the following operations: 1. `insert(word)`: Inserts the string `word` into the trie. 2. `search(word)`: Returns `true` if the string `word` is in the trie (i.e., was inserted before), and `false` otherwise. 3. `startsWith(prefix)`: Returns `true` if there is a previously inserted string `word` that has the prefix `prefix`, and `false` otherwise.

Examples:

```

Trie trie = new Trie();
trie.insert("apple");
trie.search("apple"); // returns true
trie.search("app");   // returns false
trie.startsWith("app"); // returns true

```

```
trie.insert("app");
trie.search("app");    // returns true
```

Approach: Use the basic trie implementation shown above.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end_of_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end_of_word = True

    def search(self, word: str) -> bool:
        node = self.root
        for char in word:
            if char not in node.children:
                return False
            node = node.children[char]
        return node.is_end_of_word

    def starts_with(self, prefix: str) -> bool:
        node = self.root
        for char in prefix:
            if char not in node.children:
                return False
            node = node.children[char]
        return True
```

Visual Example:

Operations:

trie = Trie() - Creates an empty trie with just a root node.

trie.insert("apple"):

Start at root

Add 'a' -> Add 'p' -> Add 'p' -> Add 'l' -> Add 'e'

Mark the node after 'e' as the end of a word

```

trie.search("apple"):
    Start at root
    Follow path: 'a' -> 'p' -> 'p' -> 'l' -> 'e'
    Check if the final node is marked as the end of a word: Yes
    Return true

trie.search("app"):
    Start at root
    Follow path: 'a' -> 'p' -> 'p'
    Check if the final node is marked as the end of a word: No
    Return false

trie.startsWith("app"):
    Start at root
    Follow path: 'a' -> 'p' -> 'p'
    Check if the path exists: Yes
    Return true

trie.insert("app"):
    Start at root
    Follow path: 'a' -> 'p' -> 'p'
    Mark the node after 'p' as the end of a word

trie.search("app"):
    Start at root
    Follow path: 'a' -> 'p' -> 'p'
    Check if the final node is marked as the end of a word: Yes
    Return true

```

Time Complexity: - insert: $O(m)$ where m is the length of the word - search: $O(m)$ where m is the length of the word - startsWith: $O(m)$ where m is the length of the prefix Space Complexity: $O(n*m)$ where n is the number of words and m is the average length

Medium: Design Add and Search Words Data Structure Problem:

Design a data structure that supports adding new words and finding if a string matches any previously added string. The data structure should support the following operations: 1. **addWord(word)**: Adds **word** to the data structure. 2. **search(word)**: Returns **true** if there is any string in the data structure that matches **word**, and **false** otherwise. A match is defined as a string that is identical to **word**, or contains exactly one character that can be replaced by any other character to make it identical to **word**. The character '.' in **word** is treated as a wildcard that can match any character.

Examples:

```
WordDictionary wordDictionary = new WordDictionary();
```

```

wordDictionary.addWord("bad");
wordDictionary.addWord("dad");
wordDictionary.addWord("mad");
wordDictionary.search("pad"); // return False
wordDictionary.search("bad"); // return True
wordDictionary.search(".ad"); // return True
wordDictionary.search("b.."); // return True

```

Approach: Extend the basic trie to handle the wildcard character during search.

```

class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end_of_word = False

class WordDictionary:
    def __init__(self):
        self.root = TrieNode()

    def add_word(self, word: str) -> None:
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end_of_word = True

    def search(self, word: str) -> bool:
        def dfs(node: TrieNode, index: int) -> bool:
            # Base case: reach the end of the word
            if index == len(word):
                return node.is_end_of_word

            char = word[index]

            # Handle wildcard
            if char == '.':
                # Try all possible characters
                for child in node.children.values():
                    if dfs(child, index + 1):
                        return True
                return False

            # Normal character
            if char not in node.children:
                return False

```

```

        return dfs(node.children[char], index + 1)

    return dfs(self.root, 0)

```

Visual Example:

Operations:

```

wordDictionary = WordDictionary() - Creates an empty trie.

wordDictionary.addWord("bad"):
    Start at root
    Add 'b' -> Add 'a' -> Add 'd'
    Mark final node as end of word

wordDictionary.addWord("dad"), wordDictionary.addWord("mad"):
    (similar process to above)

wordDictionary.search("pad"):
    Start at root, index = 0, char = 'p'
    'p' not in children, return false

wordDictionary.search("bad"):
    Start at root, index = 0, char = 'b'
    Follow path: 'b' -> 'a' -> 'd'
    Reach the end of the word, check is_end_of_word: true
    Return true

wordDictionary.search(".ad"):
    Start at root, index = 0, char = '.'
    Try all children at root: 'b', 'd', 'm'
    For 'b', continue with index = 1, char = 'a'
        Follow path: 'a' -> 'd'
        Reach the end of the word, check is_end_of_word: true
        Return true
    (No need to check 'd' and 'm' since we already found a match)

```

Time Complexity: - add_word: $O(m)$ where m is the length of the word - search: $O(n26^d)$ where n is the number of words, d is the number of wildcards (in worst case) Space Complexity: $O(nm)$ where n is the number of words and m is the average length

Medium: Replace Words Problem: In English, we have a concept called “root”, which can be followed by some other words to form another longer word - let’s call this word “successor”. For example, when the root “an” is followed by the successor word “other”, we can form a new word “another”.

Given a dictionary consisting of many roots and a sentence consisting of words

separated by spaces, replace all the successors in the sentence with the root forming it. If a successor can be replaced by more than one root, replace it with the root that has the shortest length. Return the sentence after the replacement.

Examples: - `replace_words(["cat", "bat", "rat"], "the cattle was rattled by the battery")` should return "the cat was rat by the bat"
- `replace_words(["a", "b", "c"], "aadsfasf absbs bbab cadsfafs")` should return "a a b c"

Approach: Build a trie from the dictionary, then for each word in the sentence, find the shortest prefix that is a root.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end_of_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end_of_word = True

    def find_shortest_prefix(self, word: str) -> str:
        node = self.root
        prefix = ""

        for char in word:
            if node.is_end_of_word:
                return prefix

            if char not in node.children:
                return word

            prefix += char
            node = node.children[char]

        # Check if the last character makes a complete root
        if node.is_end_of_word:
            return prefix
```

```

        return word

def replace_words(dictionary: list[str], sentence: str) -> str:
    # Build the trie
    trie = Trie()
    for root in dictionary:
        trie.insert(root)

    # Process each word in the sentence
    words = sentence.split()
    for i in range(len(words)):
        words[i] = trie.find_shortest_prefix(words[i])

    return " ".join(words)

```

Visual Example:

```
replace_words(["cat", "bat", "rat"], "the cattle was rattled by the battery"):
```

Build trie with roots: "cat", "bat", "rat"

Process words:

```

"the" -> No matching prefix, keep "the"
"cattle" -> Prefix "cat" is a root, replace with "cat"
"was" -> No matching prefix, keep "was"
"rattled" -> Prefix "rat" is a root, replace with "rat"
"by" -> No matching prefix, keep "by"
"the" -> No matching prefix, keep "the"
"battery" -> Prefix "bat" is a root, replace with "bat"

```

Result: "the cat was rat by the bat"

Time Complexity: $O(m + n \cdot k)$ - Where m is the total length of all roots, n is the number of words in the sentence, and k is the average word length
Space Complexity: $O(m)$ - For storing the trie of dictionary roots

Graphs

Theory

Graphs are collections of nodes (vertices) connected by edges. They are used to represent networks, relationships, paths, and many other structures.

Graph Types:

| Graph Type | Description |
|-------------------------|-------------------------|
| Undirected Graph | Edges have no direction |

| Graph Type | Description |
|---------------------------|---|
| Directed Graph | Edges have direction |
| Weighted Graph | Edges have weights/costs |
| Unweighted Graph | Edges have no weights |
| Cyclic Graph | Contains at least one cycle |
| Acyclic Graph | Contains no cycles |
| Connected Graph | All vertices are reachable from any vertex |
| Disconnected Graph | Has isolated components |
| Complete Graph | Every vertex connected to all others |
| Bipartite Graph | Vertices can be divided into two groups with no edges within groups |

Undirected Graph Visualization:

```

1 --- 2
|     |
|     |
4 --- 3

```

Directed Graph Visualization:

```

1 --> 2
^     |
|     v
4 <-- 3

```

Weighted Graph Visualization:

```

      5
1 --- 2
|     |
2|     |7
|     |
4 --- 3
      1

```

Graph Representation:

1. Adjacency Matrix:

Graph:

```

1 --- 2
|     |
|     |
4 --- 3

```

Adjacency Matrix:

| | 1 | 2 | 3 | 4 |
|---|--------------|---|---|---|
| 1 | [0, 1, 0, 1] | | | |
| 2 | [1, 0, 1, 0] | | | |
| 3 | [0, 1, 0, 1] | | | |
| 4 | [1, 0, 1, 0] | | | |

2. Adjacency List:

Graph: 1 --- 2
 | |
 | |
 4 --- 3

Adjacency List:

```
1: [2, 4]
2: [1, 3]
3: [2, 4]
4: [1, 3]
```

Python Implementation:

```
# Adjacency Matrix
graph_matrix = [
    [0, 1, 0, 1],
    [1, 0, 1, 0],
    [0, 1, 0, 1],
    [1, 0, 1, 0]
]

# Adjacency List using dictionary
graph_list = {
    1: [2, 4],
    2: [1, 3],
    3: [2, 4],
    4: [1, 3]
}

# Adjacency List using list of lists (0-indexed)
graph_list_array = [
    [1, 3], # Node 0 is connected to 1 and 3
    [0, 2], # Node 1 is connected to 0 and 2
    [1, 3], # Node 2 is connected to 1 and 3
    [0, 2]  # Node 3 is connected to 0 and 2
]
```

Graph Traversals:

1. Breadth-First Search (BFS):

```

from collections import deque

def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    result = []

    while queue:
        vertex = queue.popleft()
        result.append(vertex)

        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return result

```

2. Depth-First Search (DFS):

```

def dfs(graph, start):
    visited = set()
    result = []

    def dfs_recursive(vertex):
        visited.add(vertex)
        result.append(vertex)

        for neighbor in graph[vertex]:
            if neighbor not in visited:
                dfs_recursive(neighbor)

    dfs_recursive(start)
    return result

```

Common Graph Algorithms:

1. **BFS** - Find shortest path in unweighted graph
2. **DFS** - Explore all paths, detect cycles
3. **Dijkstra's** - Find shortest path with positive weights
4. **Bellman-Ford** - Find shortest path with negative weights
5. **Floyd-Warshall** - Find all-pairs shortest paths
6. **Kruskal's/Prim's** - Find minimum spanning tree
7. **Topological Sort** - Order vertices in Directed Acyclic Graph (DAG)

When to Use Graphs Use graphs when: - The problem involves relationships between entities - You need to find paths, connections, or traversals - The problem

can be modeled as nodes and edges - You need to analyze network properties or connectivity

Example Problems

Easy: Number of Islands **Problem:** Given an $m \times n$ 2D binary grid `grid` which represents a map of '1's (land) and '0's (water), write a function that returns the number of islands. An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically.

Examples:

```
Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
Output: 1
```

```
Input: grid = [
  ["1","1","0","0","0"],
  ["1","1","0","0","0"],
  ["0","0","1","0","0"],
  ["0","0","0","1","1"]
]
Output: 3
```

Approach: Use DFS to explore each island. When we find a land cell, explore all connected land cells and mark them as visited.

```
def num_islands(grid: list[list[str]]) -> int:
    if not grid or not grid[0]:
        return 0

    m, n = len(grid), len(grid[0])
    count = 0

    def dfs(row: int, col: int) -> None:
        # Check if out of bounds or not land
        if (row < 0 or row >= m or col < 0 or col >= n or
            grid[row][col] != '1'):
            return

        # Mark as visited
        grid[row][col] = '0'

        # Explore neighbors
```

```

        dfs(row + 1, col)
        dfs(row - 1, col)
        dfs(row, col + 1)
        dfs(row, col - 1)

    # Iterate through each cell in the grid
    for i in range(m):
        for j in range(n):
            if grid[i][j] == '1':
                count += 1 # Found an island
                dfs(i, j) # Explore the island

    return count

```

Visual Example:

Input grid:

```

[
    ["1","1","0","0","0"],
    ["1","1","0","0","0"],
    ["0","0","1","0","0"],
    ["0","0","0","1","1"]
]

```

DFS exploration:

1. Start at (0,0), found land
 - Mark all connected land as visited using DFS
 - This marks all land in the top left island
 - Island count = 1
2. Continue scanning, find (2,2), another land
 - Mark it as visited
 - Island count = 2
3. Continue scanning, find (3,3), another land
 - Mark it and (3,4) as visited
 - Island count = 3
4. No more unvisited land, return 3

Time Complexity: $O(mn)$ - We visit each cell at most once
 Space Complexity: $O(mn)$ - In the worst case, the entire grid is an island (for the recursion stack)

Medium: Course Schedule Problem: There are a total of `numCourses` courses you have to take, labeled from 0 to `numCourses-1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `bi` first if you want to take course `ai`. Return `true` if you can

finish all courses, or `false` otherwise.

Examples: - `can_finish(2, [[1,0]])` should return `true` (To take course 1, you need to take course 0 first, which is possible) - `can_finish(2, [[1,0], [0,1]])` should return `false` (To take course 1, you need course 0, and to take course 0, you need course 1 - a cycle exists) - `can_finish(3, [[1,0], [2,1]])` should return `true` (Take course 0, then 1, then 2)

Approach: Model the problem as a directed graph and check for cycles using DFS. If there's a cycle, it's impossible to finish all courses.

```
def can_finish(num_courses: int, prerequisites: list[list[int]]) -> bool:
    # Build adjacency list
    graph = [[] for _ in range(num_courses)]
    for course, prereq in prerequisites:
        graph[prereq].append(course)

    # Keep track of visited nodes
    # 0 = unvisited, 1 = visiting (in current path), 2 = visited (completed)
    visited = [0] * num_courses

    def has_cycle(node: int) -> bool:
        # Already fully visited this node
        if visited[node] == 2:
            return False

        # Already visiting this node - cycle detected
        if visited[node] == 1:
            return True

        # Mark as visiting
        visited[node] = 1

        # Check neighbors
        for neighbor in graph[node]:
            if has_cycle(neighbor):
                return True

        # Mark as visited
        visited[node] = 2
        return False

    # Check each course
    for course in range(num_courses):
        if has_cycle(course):
            return False
```

```
    return True
```

Visual Example:

Example: `can_finish(3, [[1,0], [2,1]])`

Build graph:

0 -> [1]

1 -> [2]

2 -> []

DFS from node 0:

visited[0] = 1 (visiting)

Check neighbor 1:

visited[1] = 1 (visiting)

Check neighbor 2:

visited[2] = 1 (visiting)

No neighbors, mark visited[2] = 2 (completed)

No more neighbors, mark visited[1] = 2 (completed)

No more neighbors, mark visited[0] = 2 (completed)

All nodes marked as completed, no cycles found.

Return true.

Time Complexity: $O(V+E)$ - Where V is the number of courses and E is the number of prerequisites
Space Complexity: $O(V+E)$ - For the graph and visited array

Medium: Redundant Connection Problem: In an undirected graph with n nodes labeled from 1 to n, a redundant connection is an edge that creates a cycle when added to the graph. Given a graph represented as an array `edges` where `edges[i] = [ui, vi]` is an edge between nodes `ui` and `vi`, write a function that returns the redundant connection. If there are multiple answers, return the redundant connection that occurs last in the input.

Examples: - `find_redundant_connection([[1,2], [1,3], [2,3]])` should return `[2,3]` - `find_redundant_connection([[1,2], [2,3], [3,4], [1,4], [1,5]])` should return `[1,4]`

Approach: Use Union-Find algorithm to detect which edge creates a cycle.

```
def find_redundant_connection(edges: list[list[int]]) -> list[int]:  
    n = len(edges)
```

```
    # Initialize parent array for Union-Find
```

```
    parent = list(range(n + 1))
```

```
    # Find function with path compression
```

```

def find(x):
    if parent[x] != x:
        parent[x] = find(parent[x])
    return parent[x]

# Union function
def union(x, y):
    parent[find(x)] = find(y)

# Process each edge
for u, v in edges:
    # If nodes are already in the same set, this edge creates a cycle
    if find(u) == find(v):
        return [u, v]

    # Otherwise, merge the sets
    union(u, v)

return []

```

Visual Example:

```
find_redundant_connection([[1,2], [2,3], [3,4], [1,4], [1,5]])
```

Initialize parent = [0, 1, 2, 3, 4, 5]

Edge [1,2]:
 find(1) = 1, find(2) = 2, different sets
 Union: parent = [0, 1, 1, 3, 4, 5]

Edge [2,3]:
 find(2) = 1, find(3) = 3, different sets
 Union: parent = [0, 1, 1, 1, 4, 5]

Edge [3,4]:
 find(3) = 1, find(4) = 4, different sets
 Union: parent = [0, 1, 1, 1, 1, 5]

Edge [1,4]:
 find(1) = 1, find(4) = 1, same set!
 This edge creates a cycle, return [1,4]

Time Complexity: $O(n)$ - Where n is the number of edges Space Complexity:

$O(n)$ - For the parent array

Backtracking

Theory

Backtracking is an algorithmic technique for solving problems recursively by trying to build a solution incrementally, and abandoning a solution (“backtracking”) as soon as it determines that the current path cannot lead to a valid solution.

Key Principles:

1. **Recursive Exploration:** Try different choices recursively
2. **Constraint Checking:** Check if the current state violates any constraints
3. **Pruning:** Abandon paths that cannot lead to valid solutions
4. **State Management:** Save and restore state when backtracking

Visualization of Backtracking (Finding paths in a maze):

S = Start, E = End, # = Wall, . = Path

Initial maze:

```
S . . #
# # . #
. . . #
# # E .
```

Exploration (showing just one possible path):

```
S → → ↓
      ↓
      ↓
      E
```

Backtracking occurs when we hit a wall or already visited cell.

Common Backtracking Template:

```
def backtrack(state, choices, result):
    # Check if solution is complete
    if is_solution(state):
        result.append(state.copy()) # Add a copy of the current solution
        return

    # Try each possible choice
    for choice in choices:
        # Check if valid choice
        if is_valid(state, choice):
            # Make the choice
            state.add(choice)
```

```

    # Recurse with updated state
    backtrack(state, choices, result)

    # Undo the choice (backtrack)
    state.remove(choice)

```

Common Backtracking Problems:

1. **Combinatorial Problems:** Permutations, combinations, subsets
2. **Constraint Satisfaction:** N-Queens, Sudoku
3. **Path Finding:** Maze solving, word search
4. **Decision Problems:** Satisfiability, coloring

When to Use Backtracking Use backtracking when: - You need to find all (or some) solutions to a problem - The problem involves making choices from a set of options - There are constraints that eliminate invalid solutions - You need to explore a search space systematically

Example Problems

Easy: Letter Combinations of a Phone Number Problem: Given a string containing digits from 2-9, write a function that returns all possible letter combinations that the number could represent (like on a telephone keypad).

Examples: - `letter_combinations("23")` should return `["ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf"]` - `letter_combinations("")` should return `[]` - `letter_combinations("2")` should return `["a", "b", "c"]`

Approach: Use backtracking to generate all combinations by making choices for each digit.

```

def letter_combinations(digits: str) -> list[str]:
    if not digits:
        return []

    # Mapping of digits to letters
    phone_map = {
        '2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
        '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'
    }

    result = []

    def backtrack(index: int, current: str) -> None:
        # If we've processed all digits, add the combination
        if index == len(digits):
            result.append(current)

```

```

        return

        # Get the letters for the current digit
        letters = phone_map[digits[index]]

        # Try each letter
        for letter in letters:
            # Add the letter and recurse
            backtrack(index + 1, current + letter)

    backtrack(0, "")
    return result

```

Visual Example:

```
letter_combinations("23"):
```

```

digits = "23", phone_map maps 2 -> "abc", 3 -> "def"
result = []

```

```

Start backtrack(0, ""):
    index = 0, digits[0] = '2', letters = "abc"

```

```

Try 'a':
    backtrack(1, "a"):
        index = 1, digits[1] = '3', letters = "def"

```

```

Try 'd':
    backtrack(2, "ad"):
        index = 2 = len(digits), add "ad" to result
        result = ["ad"]

```

```

Try 'e':
    backtrack(2, "ae"):
        index = 2 = len(digits), add "ae" to result
        result = ["ad", "ae"]

```

```

Try 'f':
    backtrack(2, "af"):
        index = 2 = len(digits), add "af" to result
        result = ["ad", "ae", "af"]

```

```

Try 'b':
    (similar process, adds "bd", "be", "bf" to result)

```

```

Try 'c':
    (similar process, adds "cd", "ce", "cf" to result)

```

```
Final result = ["ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf"]
```

Time Complexity: $O(4^n)$ where n is the number of digits (since a digit can map to at most 4 letters) Space Complexity: $O(n)$ for the recursion stack, plus $O(4^n)$ to store the result

Medium: Permutations Problem: Given an array `nums` of distinct integers, write a function that returns all possible permutations. You can return the answer in any order.

Examples: - `permute([1, 2, 3])` should return `[[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]` - `permute([0, 1])` should return `[[0,1], [1,0]]` - `permute([1])` should return `[[1]]`

Approach: Use backtracking to generate all permutations by choosing each element as the next in the sequence.

```
def permute(nums: list[int]) -> list[list[int]]:
    result = []
    n = len(nums)

    def backtrack(current: list[int], remaining: set[int]) -> None:
        # If permutation is complete
        if len(current) == n:
            result.append(current.copy())
            return

        # Try each remaining number
        for num in list(remaining):
            # Add the number to current permutation
            current.append(num)
            remaining.remove(num)

            # Recurse
            backtrack(current, remaining)

            # Backtrack (undo the choice)
            current.pop()
            remaining.add(num)

    backtrack([], set(nums))
    return result
```

Visual Example:

```
permute([1, 2, 3]):
```

```

Start backtrack([], {1, 2, 3}):
    current = [], remaining = {1, 2, 3}

    Try 1:
        current = [1], remaining = {2, 3}
        backtrack([1], {2, 3}):

            Try 2:
                current = [1, 2], remaining = {3}
                backtrack([1, 2], {3}):

                    Try 3:
                        current = [1, 2, 3], remaining = {}
                        Permutation complete, add [1, 2, 3] to result
                        Backtrack: current = [1, 2], remaining = {3}

            Backtrack: current = [1], remaining = {2, 3}

        Try 3:
            current = [1, 3], remaining = {2}
            backtrack([1, 3], {2}):

                Try 2:
                    current = [1, 3, 2], remaining = {}
                    Permutation complete, add [1, 3, 2] to result
                    Backtrack: current = [1, 3], remaining = {2}

            Backtrack: current = [1], remaining = {2, 3}

        Backtrack: current = [], remaining = {1, 2, 3}

    (Continue with Try 2 and Try 3 at the top level...)

    Final result = [[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]

Time Complexity: O(n!) - We generate all permutations Space Complexity:
O(n) - For the recursion stack and to store each permutation

```

Medium: Subsets Problem: Given an integer array `nums` of unique elements, write a function that returns all possible subsets (the power set). The solution set must not contain duplicate subsets. Return the solution in any order.

Examples: - `subsets([1, 2, 3])` should return `[[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]` - `subsets([0])` should return `[[], [0]]`

Approach: Use backtracking to generate all subsets by deciding whether to include each element.

```

def subsets(nums: list[int]) -> list[list[int]]:
    result = []
    n = len(nums)

    def backtrack(start: int, current: list[int]) -> None:
        # Add the current subset to the result
        result.append(current.copy())

        # Try to add each remaining number
        for i in range(start, n):
            # Include nums[i]
            current.append(nums[i])

            # Recurse with next position
            backtrack(i + 1, current)

            # Backtrack (exclude nums[i])
            current.pop()

    backtrack(0, [])
    return result

```

Visual Example:

subsets([1, 2, 3]):

Start backtrack(0, []):

Add [] to result, result = [[]]

For i = 0:

current = [1]

backtrack(1, [1]):

Add [1] to result, result = [[], [1]]

For i = 1:

current = [1, 2]

backtrack(2, [1, 2]):

Add [1, 2] to result, result = [[], [1], [1, 2]]

For i = 2:

current = [1, 2, 3]

backtrack(3, [1, 2, 3]):

Add [1, 2, 3] to result, result = [[], [1], [1, 2], [1, 2, 3]]

No more elements, return

Backtrack: current = [1, 2]

No more i values for this call, return

```

Backtrack: current = [1]

For i = 2:
    current = [1, 3]
    backtrack(3, [1, 3]):
        Add [1, 3] to result, result = [[], [1], [1, 2], [1, 2, 3], [1, 3]]
        No more elements, return
    Backtrack: current = [1]

    No more i values for this call, return
Backtrack: current = []

(Continue with For i = 1, 2...)

Final result = [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]
Time Complexity:  $O(n * 2^n)$  - There are  $2^n$  subsets, and we do  $O(n)$  work for each
Space Complexity:  $O(n)$  - For the recursion stack, not counting the output

```